Gráficos Interativos com Plotly

Asimov Academy

ASIMOV

Conteúdo

01. Introdução Plotly **4**
Visão Geral do Plotly . . . 4
Dash: Extensão para Dashboards . . . 4
Exemplos de Aplicações . . . 4
Objetivo do Módulo . . . 4

02. Instalação Plotly **5**
Processo de Instalação . . . 5
Passos para instalação: . . . 5
Verificando a Instalação . . . 5
Conceitos Básicos do Plotly . . . 6
Figure . . . 6
Formas de Passar Dados . . . 6

03. Utilizando o make subplots **8**
Explorando Funcionalidades Básicas do Plotly . . . 8
Criando Múltiplos Gráficos em uma Figure . . . 8
Adicionando Gráficos com add_trace . . . 9
Atualizando Propriedades da Figure . . . 10
Modificando Elementos Específicos . . . 10
Estrutura Básica para Uso do Plotly . . . 10

04. Scatter Plots **11**
Introdução aos Scatter Plots . . . 11
Implementação Básica . . . 11
Exemplo com Função Seno . . . 11
Modos de Visualização . . . 12
Mode: ‘markers’ . . . 12
Mode: ‘lines’ . . . 12
Mode: ‘lines+markers’ . . . 12
Múltiplas Séries em um Gráfico . . . 13
Gráficos de Bolha . . . 13
Personalização de Hover . . . 14

05. Bar Charts **16**
Conceitos Básicos . . . 16
Criando Gráficos de Barras Comparativos . . . 16

Interatividade dos Gráficos . . . 17
Empilhando Barras . . . 17
Personalizando Cores . . . 18
Considerações Finais . . . 18

06. Pie Charts **19**
Introdução aos Gráficos de Pizza (Pie Charts) . . . 19
Implementação Básica . . . 19
Customizações . . . 20
Hover Information . . . 20
Informações de Texto . . . 20
Tamanho da Fonte . . . 20
Cores Personalizadas . . . 21
Destacando Fatias . . . 21
Exemplo Completo com Customizações . . . 21

07. Boxplots e Histogramas **23**
Introdução aos Gráficos Estatísticos . . . 23
Histogramas . . . 23
Criando um Histograma Simples . . . 23
Comparando Distribuições . . . 24
Boxplots . . . 25
Criando um Boxplot Vertical . . . 25
Interpretando um Boxplot . . . 26
Criando um Boxplot Horizontal . . . 26
Conclusão . . . 27

08. Histogram2d e Heatmaps **28**
Gráficos para Análise de Três Dimensões . . . 28
Histograma 2D . . . 28
Implementação Básica . . . 28
Características do Histograma 2D . . . 29
Mapas de Calor (Heatmaps) . . . 29
Implementação Básica . . . 29
Características dos Mapas de Calor . . . 30
Considerações na Escolha entre Histogram2d e Heatmap . . . 31
Customizações Adicionais . . . 31

09. Candlestick **32**
Introdução . . . 32
Criando um Gráfico de Candlestick no Plotly . . . 32
Código para Criar o Gráfico . . . 32
Interpretação do Gráfico . . . 34
Ajustes no Gráfico . . . 34
Conclusão . . . 35

10. Mesh 3D **36**
Criando um Gráfico Mesh 3D . . . 36
Explicação do Código . . . 37
Personalização do Gráfico . . . 37
Aplicações do Mesh 3D . . . 37
Conclusão . . . 37

11. Plotly Express **38**
Introdução . . . 38
Importação . . . 38
Exemplo de Gráfico Scatter (Dispersão) . . . 38
Gráfico de Barras . . . 39
Scatter Matrix (Matriz de Dispersão) . . . 40
Gráfico de Bolhas . . . 41
Animação Temporal . . . 42
Sunburst (Explosão Solar) . . . 43
Heatmaps (Mapas de Densidade) . . . 44
Mapas Interativos . . . 45
Conclusão . . . 45

01. Introdução Plotly

Visão Geral do Plotly

Plotly é uma das mais poderosas bibliotecas de visualização de dados para Python atualmente. Esta biblioteca se destaca principalmente por sua interatividade, permitindo a criação de gráficos dinâmicos e complexos para análise e apresentação de dados.

O site oficial do Plotly pode ser acessado clicando aqui, onde é possível encontrar documentação e exemplos dos diversos tipos de gráficos disponíveis na biblioteca.

Uma das principais vantagens do Plotly é sua excelente integração com o JupyterLab, proporcionando visualizações sincronizadas e interativas durante apresentações de ciência de dados.

Dash: Extensão para Dashboards

O ecossistema Plotly inclui o Dash, uma biblioteca auxiliar desenvolvida pela mesma equipe. O Dash permite a criação de dashboards interativos utilizando apenas Python, incorporando todos os gráficos desenvolvidos com o Plotly.

Exemplos de Aplicações

Com o Dash e o Plotly, é possível desenvolver diversos tipos de projetos, como:

- Análise de vídeos com Machine Learning em tempo real
- Detecção de seres humanos em vídeos e imagens
- Estimação de objetos em imagens
- Dashboards corporativos para relatórios de vendas
- Gráficos de evolução de faturamento
- Aplicações web interativas diversas

Objetivo do Módulo

Este módulo foi criado para facilitar o aprendizado desta poderosa biblioteca, considerando que sua documentação oficial pode ser desafiadora para iniciantes. O conteúdo busca extrair a essência da biblioteca Plotly e tornar o processo de aprendizado mais acessível.

02. Instalação Plotly

Processo de Instalação

O Plotly pode ser instalado em diferentes sistemas operacionais (Windows, Mac, Linux) utilizando o gerenciador de pacotes PIP. Recomenda-se utilizar o Anaconda, pois ele unifica o processo de instalação em todos os sistemas operacionais.

Passos para instalação:

Para usuários de Linux e Mac:

1. Abrir o terminal
2. Executar o comando:

   ```
   pip3 install plotly
   ```

Para usuários de Windows:

1. Abrir o Anaconda Prompt
2. Executar o comando:

   ```
   pip3 install plotly
   ```

Além do Plotly, é necessário instalar o IPywidgets, um plugin que permite executar as visualizações do Plotly no Jupyter Notebook:

```
pip3 install ipywidgets
```

Verificando a Instalação

Para verificar se o Plotly e o IPywidgets foram instalados corretamente, é possível executar um gráfico básico no Jupyter Notebook:

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Bar(x=[1, 2, 3], y=[1, 3, 2])],
    layout=go.Layout(title="Meu primeiro gráfico com Plotly")
)
fig.show()
```

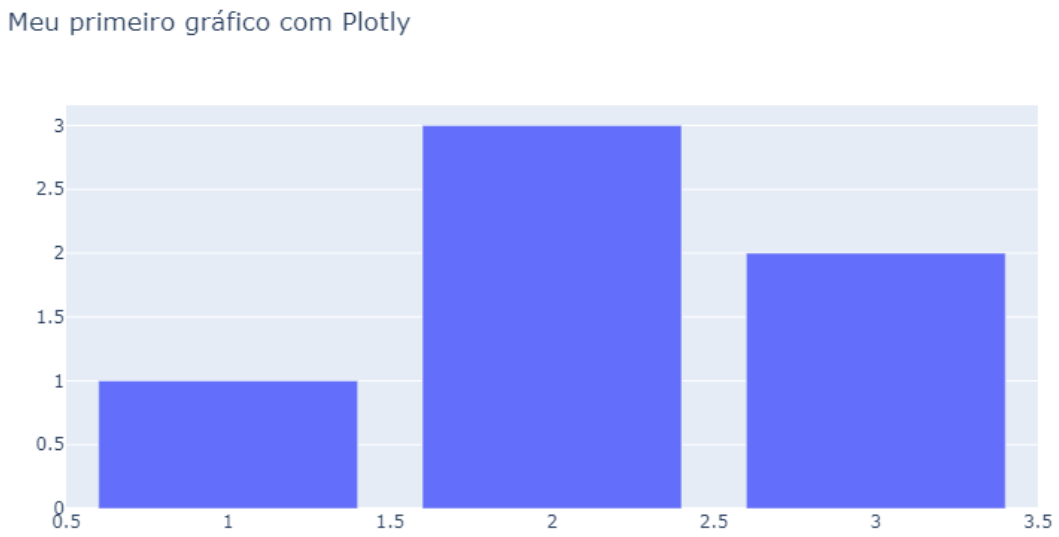

Figure 1: First Graph Plotly

Se o gráfico for exibido corretamente, significa que a instalação foi bem-sucedida.

Conceitos Básicos do Plotly

A biblioteca Plotly é bastante ampla, mas é importante compreender sua estrutura básica para utilizar seus recursos de forma eficiente.

Figure

O objeto principal do Plotly é o `Figure`. Ele funciona como um contêiner onde todos os gráficos são inseridos. Uma Figure possui três atributos principais:

1. **Data**: Contém os dados que serão plotados.
2. **Layout**: Define as configurações de aparência do gráfico (título, tamanho, etc.).
3. **Frames**: Utilizado em casos específicos para criar animações (raramente utilizado).

Formas de Passar Dados

Existem duas formas principais de passar dados para o Plotly:

1. **Dicionários**: Seguindo especificações da biblioteca.
2. **Objetos do Plotly**: Mais intuitivos e fáceis de utilizar (abordagem recomendada).

O exemplo apresentado utiliza a segunda abordagem, com `go.Figure` e `go.Bar`, que são objetos do Plotly que facilitam a criação de gráficos.

03. Utilizando o make subplots

Explorando Funcionalidades Básicas do Plotly

Após a criação do primeiro gráfico com o Plotly, é importante explorar as funcionalidades disponíveis na interface. Todo gráfico gerado apresenta uma barra superior com diversas opções:

- Download da imagem em formato PNG
- Ferramentas de zoom para áreas específicas
- Marcações ao posicionar o cursor sobre o gráfico (customizáveis)
- Ferramenta Pan para navegação
- Caixa de seleção para interagir com elementos do gráfico
- Opção Last Select para seleções específicas
- Zoom In e Zoom Out
- Autoscale para encontrar automaticamente a melhor representação
- Opção para voltar ao eixo original
- Funcionalidades para ligar/desligar linhas no Spike
- Exibição do dado mais próximo à seleção
- Comparação de dados na seleção

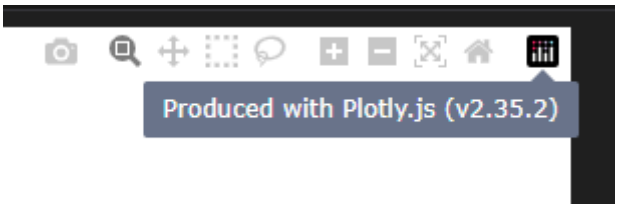

Figure 2: Ferramentas Plotly

Estas funcionalidades são incluídas automaticamente em qualquer gráfico criado com o Plotly.

Criando Múltiplos Gráficos em uma Figure

Para criar múltiplos gráficos dentro de uma mesma figure, é necessário utilizar a função `make_subplots`. Primeiro, deve-se importar esta função:

```
from plotly.subplots import make_subplots
```

Em seguida, pode-se criar uma figure utilizando esta função:

```
fig = make_subplots(rows=1, cols=2)
```

O parâmetro `rows` especifica quantas linhas a figure terá, e `cols` especifica quantas colunas. No exemplo acima, a figure terá 1 linha e 2 colunas.

Adicionando Gráficos com add_trace

Para adicionar gráficos à figure, utiliza-se o método `add_trace`:

```
fig.add_trace(
    go.Bar(
        y=[1, 4, 5],
        x=[6, 5, 2],
        marker_color='green'
    ),
    row=1, col=1
)

fig.add_trace(
    go.Scatter(
        y=[1, 4, 5],
        x=[6, 5, 2],
        mode='lines'
    ),
    row=1, col=2
)
```

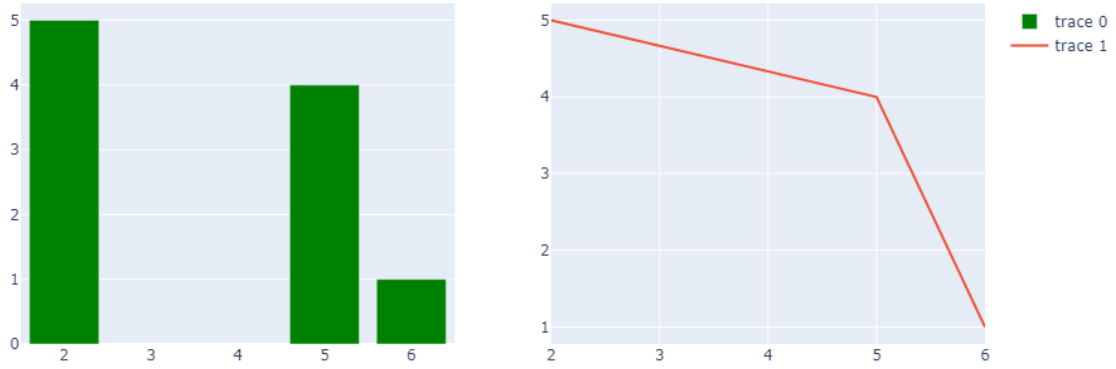

Figure 3: Segundo Gráfico em Coluna

No código acima:

- O primeiro `add_trace` adiciona um gráfico de barras na posição (1,1)

- O segundo `add_trace` adiciona um gráfico de linhas na posição (1,2)

O gráfico `Scatter` pode ser configurado de diferentes formas, alterando o parâmetro `mode`:

- `mode='lines'`: exibe linhas conectando os pontos
- `mode='markers'`: exibe apenas os pontos (sem linhas)
- Se o parâmetro mode for omitido, o Plotly exibirá por padrão:

  `lines+markers` (linhas e pontos), se houver menos de 20 pontos e o gráfico não estiver empilhado. Apenas `lines` (linhas sem pontos), se houver 20 ou mais pontos ou se o gráfico estiver empilhado.

Atualizando Propriedades da Figure

Após criar uma figure, é possível atualizar suas propriedades utilizando o método `update_layout`:

```
fig.update_layout(
    title_text="Meu Gráfico",
    title_font_size=20
)
```

Também é possível alterar as dimensões da figure:

```
fig.update_layout(height=700)
```

Modificando Elementos Específicos

Para modificar elementos específicos de um gráfico, é possível acessar diretamente os dados:

```
fig.data[0].marker.line.width = 4
fig.data[0].marker.line.color = 'black'
```

No exemplo acima: - `fig.data[0]` acessa o primeiro gráfico adicionado à figure - `marker.line.width` define a espessura da linha - `marker.line.color` define a cor da linha

Estrutura Básica para Uso do Plotly

A estrutura geral para criar visualizações com Plotly segue estes passos:

1. Criar uma figure (com `go.Figure()` ou `make_subplots()`)
2. Adicionar gráficos à figure com `add_trace()`
3. Customizar a figure com `update_layout()`
4. Exibir a figure com `fig.show()`

Esta estrutura básica se aplica a todos os tipos de gráficos disponíveis no Plotly.

04. Scatter Plots

Introdução aos Scatter Plots

Após compreender os conceitos básicos do Plotly, é possível estudar as especificidades de cada tipo de gráfico. Os Scatter Plots são gráficos de pontos ou linhas, que podem ser intercambiados conforme a necessidade da visualização.

Implementação Básica

Para criar um Scatter Plot, é necessário importar os módulos necessários:

```
import plotly.graph_objects as go
import numpy as np
```

A comunidade do Plotly adota o padrão de nomear o módulo `plotly.graph_objects` como go, o que será seguido nos exemplos.

Exemplo com Função Seno

Primeiro, é necessário criar dados para visualização:

```
# Criando um vetor com 100 pontos uniformemente distribuídos entre 0 e 10
t = np.linspace(0, 10, 100)
# Aplicando a função seno aos valores de t
y = np.sin(t)
```

Em seguida, é possível criar um Scatter Plot básico:

```
# Criando figura com pontos (markers)
fig = go.Figure(
    data=[go.Scatter(x=t, y=y, mode='markers')]
)
fig.show()
```

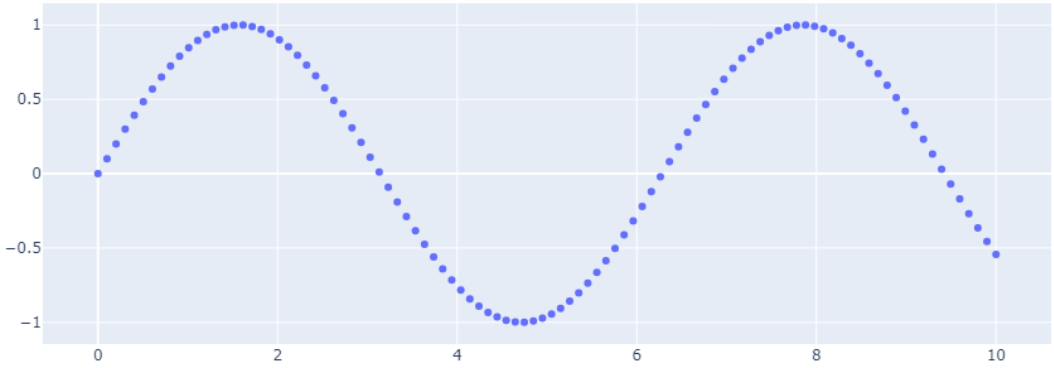

Figure 4: Scatter Graph Plotly

Modos de Visualização

O parâmetro `mode` define como os dados serão exibidos no gráfico:

Mode: 'markers'

Exibe apenas pontos, útil quando não há relação linear clara entre as variáveis.

```
go.Scatter(x=t, y=y, mode='markers')
```

Mode: 'lines'

Conecta os pontos com linhas, ideal para mostrar tendências ou relações lineares.

```
go.Scatter(x=t, y=y, mode='lines')
```

Mode: 'lines+markers'

Exibe tanto pontos quanto linhas que os conectam.

```
go.Scatter(x=t, y=y, mode='lines+markers')
```

Múltiplas Séries em um Gráfico

É possível adicionar múltiplas séries de dados em um único gráfico:

```
# Criando dados para exemplo
n = 100
x = np.linspace(0, 1, n)
y0 = x + 5 + np.random.randn(n)
y1 = x + np.random.randn(n)
y2 = x - 5 + np.random.randn(n)

# Criando figura
fig = go.Figure()

# Adicionando séries de dados
fig.add_trace(go.Scatter(x=x, y=y0, mode='markers', name='markers'))
fig.add_trace(go.Scatter(x=x, y=y1, mode='lines+markers', name='lines+markers'))
fig.add_trace(go.Scatter(x=x, y=y2, mode='lines', name='lines'))

fig.show()
```

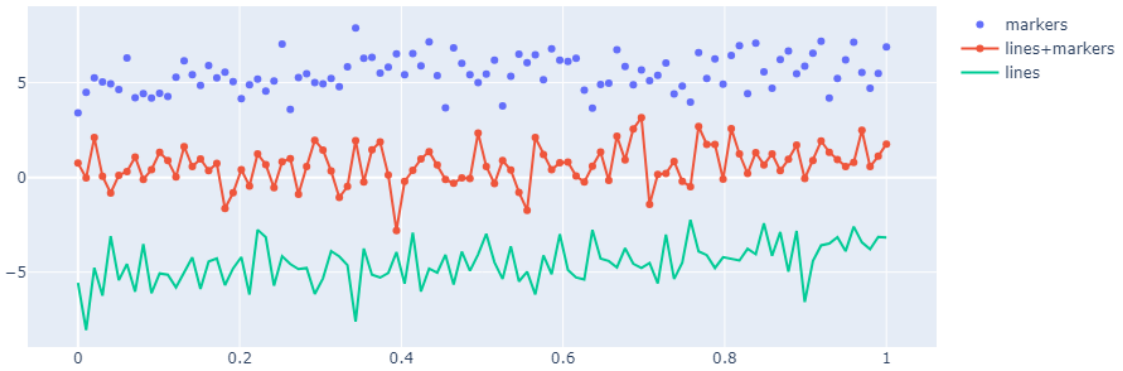

Figure 5: Scatter Trace Exemplos

O parâmetro `name` adiciona automaticamente uma legenda para identificar cada série.

Gráficos de Bolha

É possível criar gráficos de bolha (bubble charts) configurando o atributo `marker`:

```
fig = go.Figure(data=[go.Scatter(
    x=[1, 2, 3, 4],
    y=[10, 11, 12, 13],
    mode='markers',
    marker=dict(
        size=[40, 60, 80, 100],
        color=[0, 1, 2, 3]
    )
)])
fig.show()
```

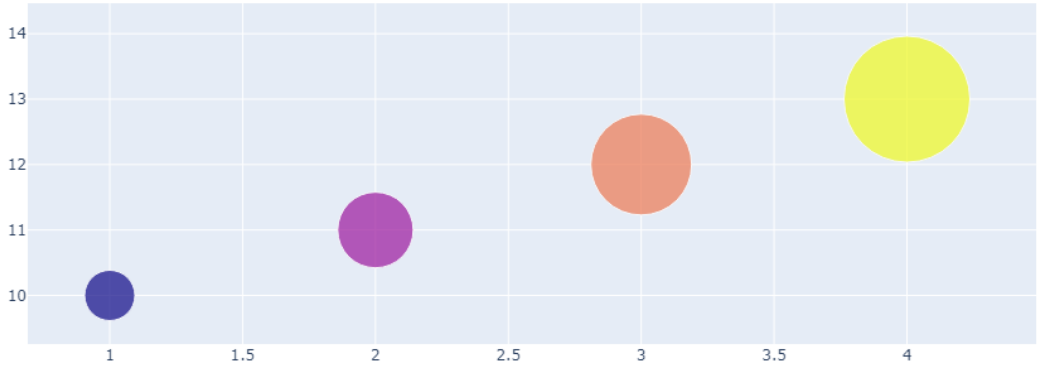

Figure 6: Bubble Chart Exemplo

O atributo `size` define o tamanho dos marcadores, sendo necessário que o vetor tenha o mesmo tamanho dos vetores x e y. O atributo `color` é opcional e define a cor de cada marcador.

Personalização de Hover

Por padrão, ao passar o cursor sobre um ponto, são exibidos os valores de x e y. É possível personalizar essa informação com o parâmetro `hovertemplate`:

```
fig = go.Figure(data=[go.Scatter(
    x=[1, 2, 3, 4],
    y=[10, 11, 12, 13],
    mode='markers',
    marker=dict(
        size=[40, 60, 80, 100]
    ),
    hovertemplate='R$%{y}<br>Tamanho: %{marker.size}'
```

```
)])
fig.show()
```

O formato `%{variável}` permite acessar valores específicos como y ou marker.size. A documentação do Plotly lista todas as variáveis disponíveis para uso no hovertemplate.

05. Bar Charts

Conceitos Básicos

Os gráficos de barras (bar charts) são uma das visualizações mais comuns e úteis para representar dados categóricos. Esta seção explora com mais detalhes este tipo de gráfico no Plotly, expandindo os conceitos básicos apresentados anteriormente.

Criando Gráficos de Barras Comparativos

Para criar um gráfico de barras comparativo que mostra dados de diferentes categorias, é necessário primeiro importar a biblioteca:

```
import plotly.graph_objects as go
```

O exemplo a seguir demonstra como criar um gráfico comparando a quantidade de animais em dois zoológicos diferentes:

```
# Definindo as categorias
animais = ["Girafas", "Macacos", "Tigres"]

# Criando a figura
fig = go.Figure(
    data=[
        go.Bar(
            x=animais,
            y=[3, 120, 6],
            name="Zoo de São Paulo"
        ),
        go.Bar(
            x=animais,
            y=[1, 40, 2],
            name="Zoo do Rio Grande do Sul"
        )
    ]
)

# Exibindo o gráfico
fig.show()
```

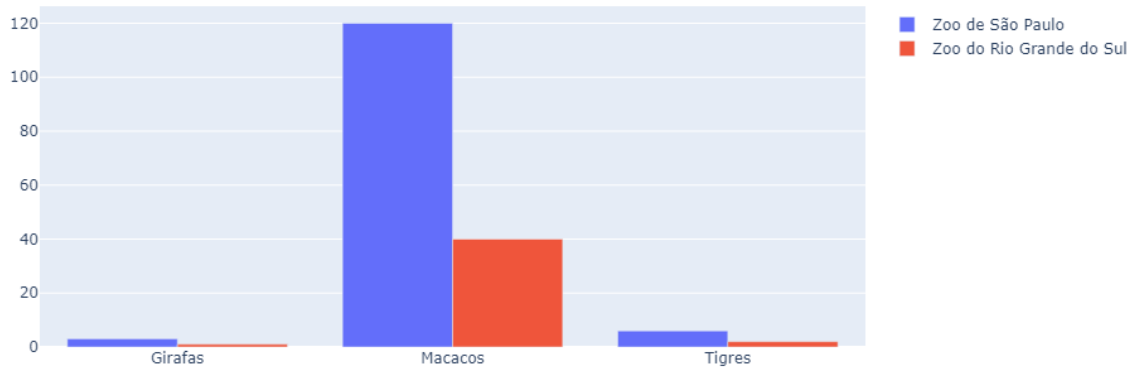

Figure 7: Gráfico Zoológico

Neste exemplo:

- x contém as categorias (tipos de animais)
- y contém os valores (quantidade de cada animal)
- name identifica cada série de dados (zoológicos diferentes)

Interatividade dos Gráficos

Os gráficos do Plotly possuem funcionalidades interativas incorporadas:

- Clicar na legenda permite habilitar/desabilitar séries específicas
- Passar o cursor sobre as barras exibe os valores correspondentes
- Ferramentas de zoom permitem visualizar áreas específicas do gráfico

Empilhando Barras

É possível modificar a disposição das barras para que sejam empilhadas ao invés de exibidas lado a lado:

```
fig.update_layout(barmode="stack")
```

Esta visualização facilita a comparação da soma total das categorias, além de manter a visualização das contribuições individuais.

Personalizando Cores

A personalização das cores das barras pode ser feita de diferentes maneiras:

```
# Definindo uma lista de cores
cores = ["lightgray"] * 5  # Cria uma lista com 5 elementos "lightgray"

# Criando o gráfico com cores personalizadas
fig = go.Figure(
    data=[
        go.Bar(
            x=["A", "B", "C", "D", "E"],
            y=[10, 20, 15, 30, 25],
            marker_color=cores
        )
    ]
)

# Destacando uma barra específica
cores[1] = "blue"  # Altera apenas o segundo elemento para azul
fig = go.Figure(
    data=[
        go.Bar(
            x=["A", "B", "C", "D", "E"],
            y=[10, 20, 15, 30, 25],
            marker_color=cores
        )
    ]
)
```

Este método permite destacar elementos específicos do gráfico, direcionando a atenção do espectador para informações importantes.

Considerações Finais

Os gráficos de barras no Plotly oferecem uma combinação de simplicidade e flexibilidade que os torna adequados para diversas aplicações. As funcionalidades apresentadas são apenas uma introdução às possibilidades disponíveis na biblioteca. Para aplicações mais avançadas e personalizações específicas, a documentação oficial do Plotly contém informações detalhadas sobre todos os parâmetros disponíveis.

06. Pie Charts

Introdução aos Gráficos de Pizza (Pie Charts)

Os gráficos de pizza (também conhecidos como gráficos de torta em algumas regiões) são uma forma eficaz de representar proporções de um total. Este tipo de gráfico é especialmente útil para mostrar a distribuição percentual de categorias que compõem um conjunto de dados.

Implementação Básica

Para criar um gráfico de pizza com Plotly, é necessário definir os rótulos (labels) e os valores correspondentes:

```
import plotly.graph_objects as go

# Dados de exemplo
gases = ['Oxigênio', 'Nitrogênio', 'Gás Carbônico']
valores = [21, 78, 0.04]

# Criando o gráfico
fig = go.Figure(
    data=[go.Pie(labels=gases, values=valores)]
)
fig.show()
```

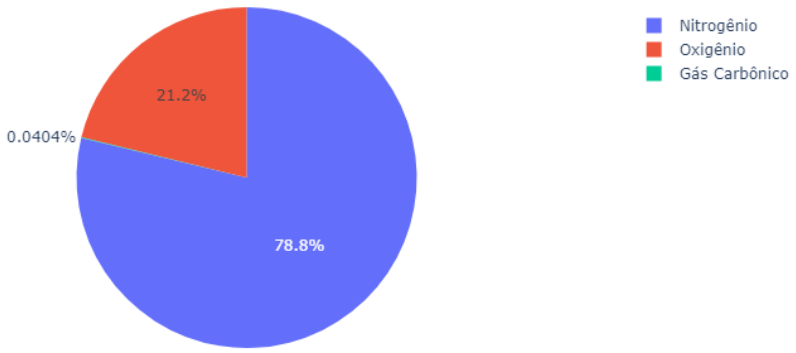

Figure 8: Exemplo Gráfico de Pizza

Este código cria um gráfico básico de pizza que mostra a proporção de cada gás na amostra. O Plotly calcula automaticamente as porcentagens com base nos valores fornecidos.

Customizações

O gráfico de pizza pode ser customizado de várias maneiras para melhorar a visualização e destacar informações importantes.

Hover Information

É possível controlar quais informações são exibidas quando o cursor passa sobre cada fatia do gráfico:

```
# Exibindo apenas o rótulo
fig.update_traces(hoverinfo='label')

# Exibindo rótulo e percentual
fig.update_traces(hoverinfo='label+percent')

# Exibindo valor e percentual
fig.update_traces(hoverinfo='value+percent')
```

Informações de Texto

As informações exibidas diretamente no gráfico também podem ser personalizadas:

```
# Exibindo valores
fig.update_traces(textinfo='value')

# Exibindo rótulos
fig.update_traces(textinfo='label')

# Exibindo percentuais
fig.update_traces(textinfo='percent')

# Combinando informações
fig.update_traces(textinfo='value+percent')
fig.update_traces(textinfo='label+percent')
```

Tamanho da Fonte

O tamanho do texto pode ser ajustado para melhorar a legibilidade:

```
fig.update_traces(textfont_size=12)
```

Cores Personalizadas

As cores das fatias do gráfico podem ser especificadas:

```
fig.update_traces(marker=dict(colors=['green', 'rgb(0,0,255)', 'pink']))
```

Destacando Fatias

É possível destacar uma fatia específica afastando-a do centro do gráfico:

```
# Destacando a terceira fatia (índice 2) em 20%
fig.update_traces(pull=[0, 0, 0.2])
```

O vetor `pull` deve ter o mesmo tamanho do vetor de valores, indicando quanto cada fatia deve ser afastada do centro. Os valores típicos estão entre 0 (sem destaque) e 1 (destaque máximo).

Exemplo Completo com Customizações

```
import plotly.graph_objects as go

# Dados de exemplo
gases = ['Oxigênio', 'Nitrogênio', 'Gás Carbônico']
valores = [21, 78, 0.04]

# Criando o gráfico
fig = go.Figure(
    data=[go.Pie(
        labels=gases,
        values=valores,
        textinfo='value+percent',
        textfont_size=12,
        marker=dict(colors=['green', 'blue', 'pink']),
        pull=[0, 0, 0.2],  # Destacando Gás Carbônico
        hoverinfo='label+percent'
    )]
)
fig.show()
```

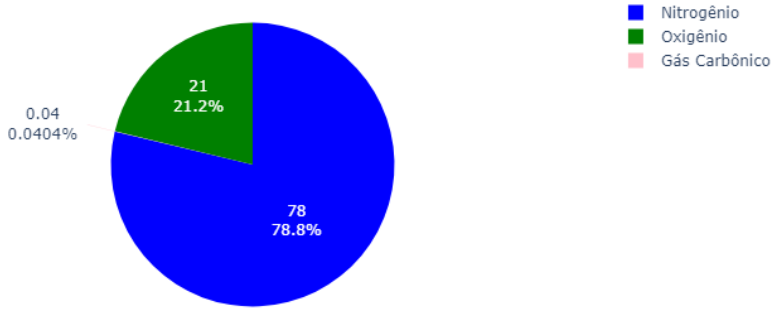

Figure 9: Exemplo Gráfico de Pizza Atualizado

Este exemplo cria um gráfico de pizza com:

- Valores e percentuais exibidos em cada fatia
- Tamanho de fonte ajustado para 12
- Cores personalizadas para cada fatia
- A fatia do Gás Carbônico destacada do centro
- Informações de rótulo e percentual exibidas ao passar o cursor

07. Boxplots e Histogramas

Introdução aos Gráficos Estatísticos

O Plotly oferece uma grande variedade de gráficos estatísticos que permitem uma análise mais metódica e científica dos dados. Estes gráficos são ferramentas essenciais para analistas de dados e para qualquer pessoa que deseje compreender a distribuição dos dados em um projeto. Entre estes gráficos, destacam-se os histogramas e boxplots, que serão abordados nesta seção.

Histogramas

Os histogramas são gráficos estatísticos que mostram a distribuição de frequência de um conjunto de dados, agrupando-os em intervalos.

Criando um Histograma Simples

Para criar um histograma, primeiro é necessário importar as bibliotecas e gerar dados para análise:

```
import plotly.graph_objects as go
import numpy as np

# Criando dados aleatórios com distribuição normal
x = np.random.randn(500)

# Criando a figura com o histograma
fig = go.Figure(
    data=[go.Histogram(x=x)]
)

# Exibindo o gráfico
fig.show()
```

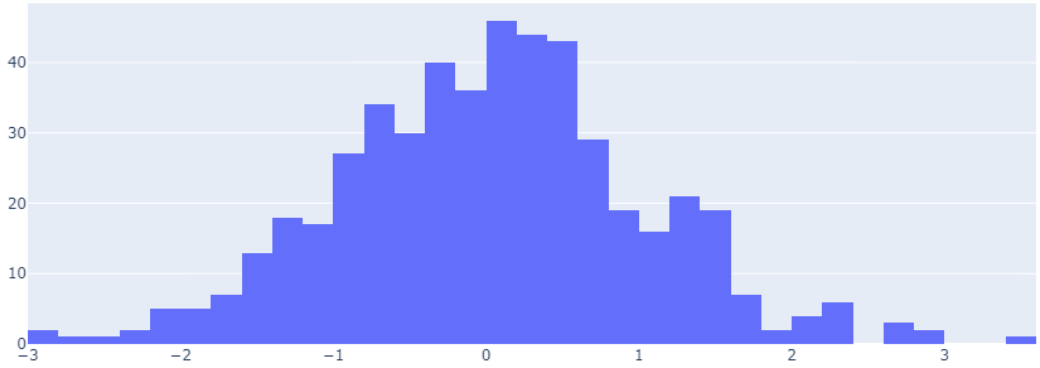

Figure 10: Exemplo de Histogram Graph

Este código cria um histograma que mostra a frequência de valores em diferentes intervalos. Ao passar o cursor sobre as barras, é possível ver o intervalo específico e quantas ocorrências existem nesse intervalo.

Comparando Distribuições

Para comparar duas distribuições diferentes no mesmo histograma:

```
# Criando um segundo conjunto de dados
x1 = np.random.randn(500) + 1  # Deslocado 1 unidade para a direita

# Criando o histograma comparativo
fig = go.Figure()
fig.add_trace(go.Histogram(x=x))
fig.add_trace(go.Histogram(x=x1))

# Sobrepondo os histogramas
fig.update_layout(barmode='overlay')

# Ajustando a opacidade para melhor visualização
fig.update_traces(opacity=0.75)

fig.show()
```

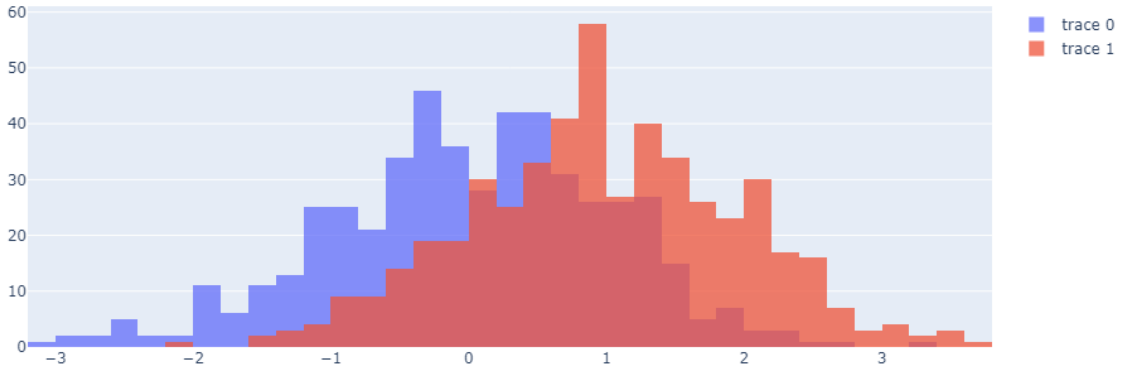

Figure 11: Exemplo de Histogram Graph Combinado

A opção `barmode='overlay'` permite sobrepor os histogramas, e ajustar a opacidade facilita a visualização de ambas as distribuições simultaneamente.

Boxplots

Boxplots são representações gráficas que mostram a distribuição dos dados através de quartis, proporcionando uma visão resumida da dispersão dos valores.

Criando um Boxplot Vertical

```
# Criando dois conjuntos de dados para comparação
y0 = np.random.randn(50) - 1  # Deslocado 1 unidade para a esquerda
y1 = np.random.randn(50) + 1  # Deslocado 1 unidade para a direita

# Criando a figura com boxplots
fig = go.Figure()
fig.add_trace(go.Box(y=y0))
fig.add_trace(go.Box(y=y1))

# Ajustando a altura para melhor visualização
fig.update_layout(height=600)

fig.show()
```

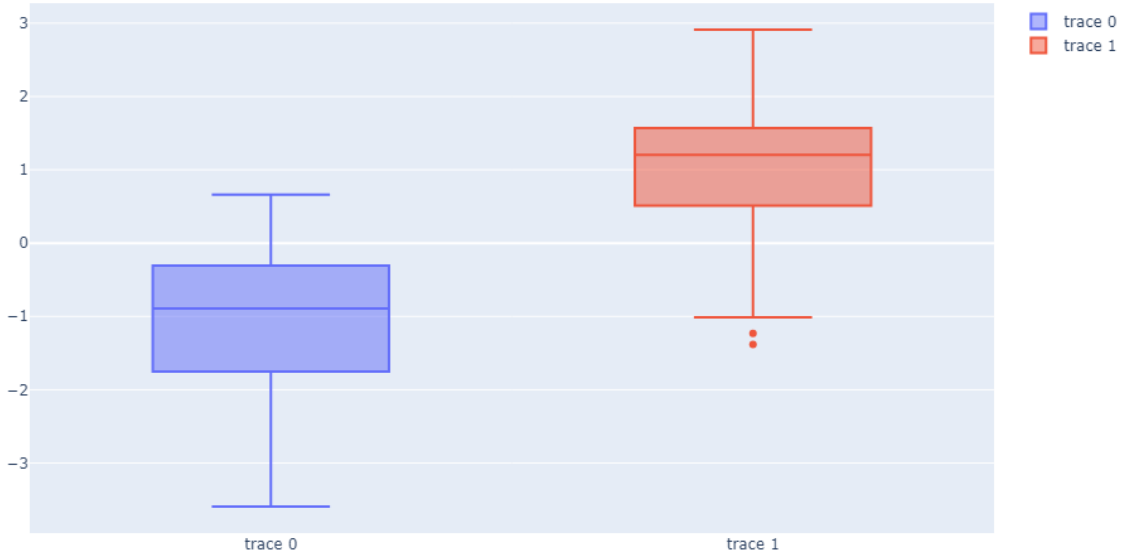

Figure 12: Exemplo de Boxplot Graph

Interpretando um Boxplot

Um boxplot apresenta as seguintes informações:

- A linha inferior (base) representa o primeiro quartil (25% dos dados)
- A linha do meio dentro da caixa representa a mediana (50% dos dados)
- A linha superior da caixa representa o terceiro quartil (75% dos dados)
- As linhas que se estendem da caixa (whiskers) mostram a amplitude dos dados
- Pontos além dos whiskers representam outliers

A caixa contém 50% dos dados, divididos pela mediana. Esta visualização permite identificar facilmente a centralidade, dispersão e possíveis outliers nos dados.

Criando um Boxplot Horizontal

Para criar um boxplot horizontal, basta utilizar o parâmetro x em vez de y:

```
# Criando boxplots horizontais
fig = go.Figure()
```

```
fig.add_trace(go.Box(x=y0))
fig.add_trace(go.Box(x=y1))

fig.update_layout(height=400)

fig.show()
```

A representação horizontal geralmente oferece melhor visualização, especialmente quando a tela tem mais largura do que altura.

Conclusão

Histogramas e boxplots são ferramentas poderosas para análise exploratória de dados, cada uma oferecendo perspectivas diferentes sobre a distribuição. Enquanto histogramas mostram a frequência de valores em diferentes intervalos, boxplots proporcionam uma visão resumida baseada em quartis. A escolha entre um ou outro depende do tipo de análise desejada e das características específicas dos dados.

08. Histogram2d e Heatmaps

Gráficos para Análise de Três Dimensões

Os gráficos de três dimensões são ferramentas poderosas para visualizar relações complexas entre variáveis. Nesta seção, serão apresentados dois tipos específicos: o Histograma 2D e os Mapas de Calor (Heatmaps).

Histograma 2D

O Histograma 2D permite visualizar a distribuição conjunta de duas variáveis contínuas. Embora o nome indique apenas duas dimensões, na verdade trabalha com três: as duas variáveis nos eixos x e y, e a contagem (frequência) representada pela intensidade da cor.

Implementação Básica

```
import plotly.graph_objects as go
import numpy as np

# Criando dados aleatórios para exemplo
x = np.random.randn(500)
y = np.random.randn(500) + 1  # Deslocado em +1 para diferenciar

# Criando o histograma 2D
fig = go.Figure(
    data=[go.Histogram2d(x=x, y=y)]
)
fig.show()
```

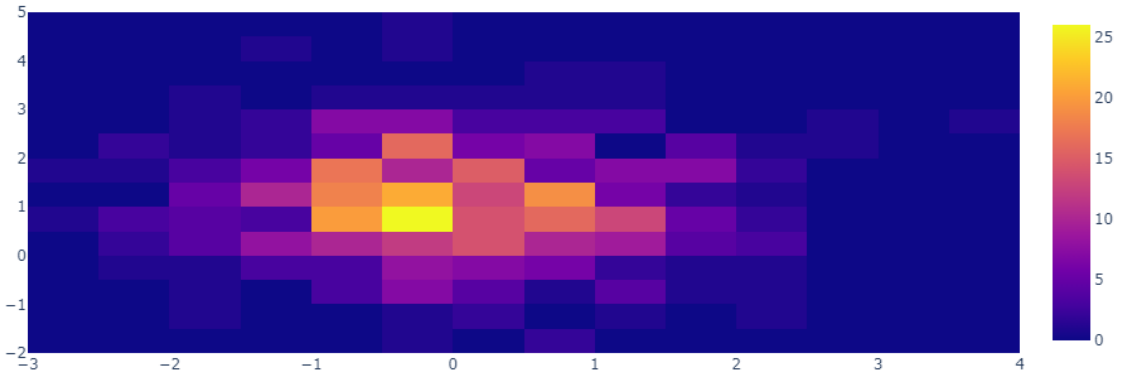

Figure 13: Exemplo de Gráfico Histogram2d

No gráfico resultante, a intensidade da cor representa a frequência com que determinada combinação de valores x e y aparece nos dados. Áreas mais claras indicam maior concentração de pontos.

Características do Histograma 2D

- Ideal para variáveis contínuas
- A terceira dimensão (cor) representa a contagem/frequência
- Útil para identificar padrões de correlação entre duas variáveis
- Auxilia na detecção de clusters e outliers

Mapas de Calor (Heatmaps)

Os Mapas de Calor são semelhantes ao Histograma 2D, mas possuem duas diferenças fundamentais:

1. Trabalham com valores categóricos ou discretos nos eixos x e y
2. Os valores da terceira dimensão (cor) são especificados diretamente, não representando necessariamente uma contagem

Implementação Básica

```
import plotly.graph_objects as go
```

```
# Definindo dados categóricos
dias = ['Segunda', 'Terça', 'Quarta', 'Quinta', 'Sexta']
turnos = ['Manhã', 'Tarde', 'Noite']

# Matriz de valores para a intensidade (terceira dimensão)
valores = [
    [25, 15, 30, 35, 25],
    [45, 30, None, 40, 20],
    [20, 10, 30, 25, 15]
]

# Criando o mapa de calor
fig = go.Figure(
    data=[go.Heatmap(
        z=valores,
        x=dias,
        y=turnos
    )]
)
fig.show()
```

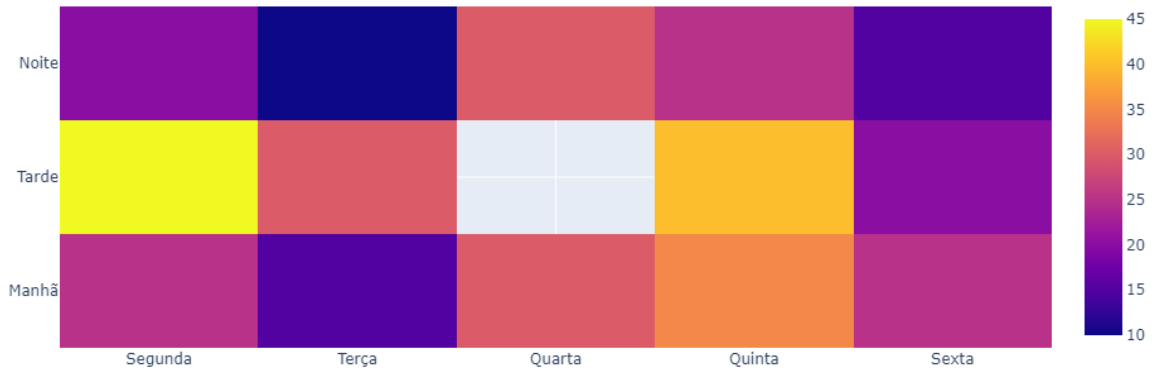

Figure 14: Exemplo de Gráfico Histogram2d Quadrado

Neste exemplo, o mapa de calor mostra a distribuição de alguma métrica (poderia ser número de visitantes, vendas, etc.) ao longo dos dias da semana e turnos.

Características dos Mapas de Calor

- Trabalham com variáveis categóricas ou discretas

- A intensidade da cor representa valores especificados, não necessariamente contagens
- Permite visualizar valores ausentes (None)
- Útil para análise de padrões em dados tabulares

Considerações na Escolha entre Histogram2d e Heatmap

- **Use Histogram2d quando:**
  - As variáveis são contínuas
  - O interesse está na frequência/distribuição conjunta
  - Os dados são numerosos e potencialmente agrupados
- **Use Heatmap quando:**
  - As variáveis são categóricas ou discretas
  - Os valores da terceira dimensão são conhecidos (não são contagens)
  - É necessário visualizar dados em formato de matriz/tabela

Customizações Adicionais

Ambos os gráficos aceitam diversos parâmetros de customização disponíveis na documentação do Plotly, incluindo:

- Escalas de cores
- Formatação de texto
- Ajustes de bins (no caso do Histogram2d)
- Controle de valores mínimos e máximos
- Anotações e títulos

09. Candlestick

Introdução

Candlestick é um tipo de gráfico muito comum para analisar dados financeiros. Ele permite visualizar como a cotação de ativos no mercado financeiro variou ao longo do tempo.

Antes de explorar seu funcionamento, será gerado um gráfico para exemplificação. Os dados utilizados representam a cotação das ações da Apple de 2023, incluindo:

- Preço de abertura
- Preço máximo do dia
- Preço mínimo do dia
- Preço de fechamento
- Volume negociado

Com essas informações, é possível criar um gráfico de candlestick para representar a evolução dos preços ao longo do tempo.

Criando um Gráfico de Candlestick no Plotly

No Plotly, a estrutura para criação de gráficos segue um padrão, variando apenas conforme as configurações específicas de cada tipo de gráfico.

Código para Criar o Gráfico

```
import plotly.graph_objects as go
import pandas as pd
import random

# Gerando dados fictícios para exibição
import numpy as np

np.random.seed(42)
num_days = 100

dates = pd.date_range(start="2023-01-01", periods=num_days, freq="D")
open_prices = np.round(np.random.uniform(90, 130, num_days), 2)
close_prices = open_prices + np.round(np.random.uniform(-5, 5, num_days), 2)
high_prices = np.maximum(open_prices, close_prices) + np.round(np.random.uniform(0, 2,
↪  num_days), 2)
low_prices = np.minimum(open_prices, close_prices) - np.round(np.random.uniform(0, 2,
↪  num_days), 2)

# Criar DataFrame
```

```
df = pd.DataFrame({
    "Date": dates,
    "Open": open_prices,
    "High": high_prices,
    "Low": low_prices,
    "Close": close_prices
})

# Criando o gráfico de candlestick
fig = go.Figure(
    data=[
        go.Candlestick(
            x=df["Date"],
            open=df["Open"],
            high=df["High"],
            low=df["Low"],
            close=df["Close"],
            increasing_line_color="green",  # Velas de alta (fechamento > abertura)
            decreasing_line_color="red"  # Velas de baixa (fechamento < abertura)
        )
    ]
)

# Ajustando layout para melhor visualização
fig.update_layout(
    title="Gráfico de Candlestick - Exemplo",
    xaxis_title="Data",
    yaxis_title="Preço",
    xaxis_rangeslider_visible=False,  # Removendo slider
    template="plotly_white"  # Fundo mais limpo
)

# Exibir gráfico
fig.show()
```

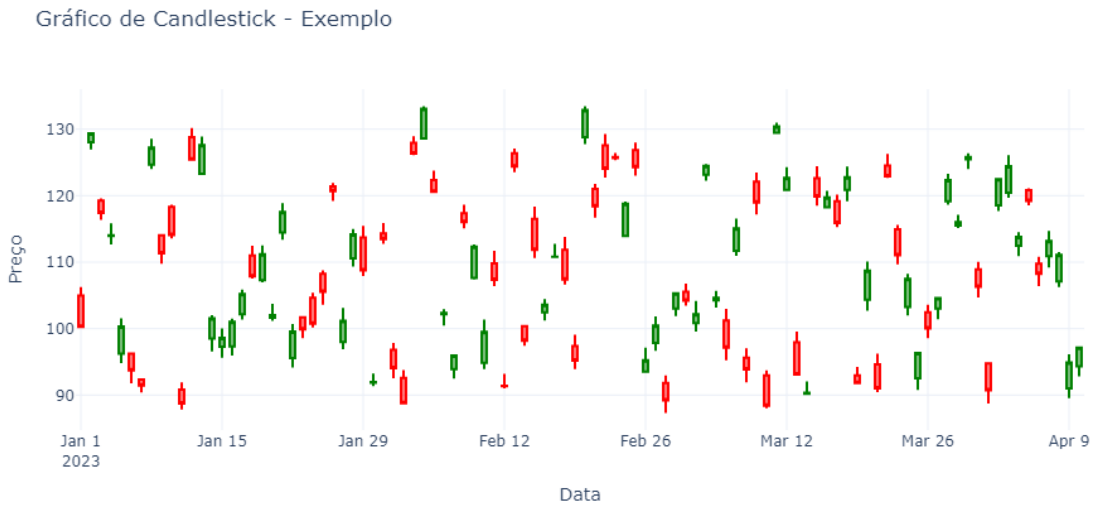

Figure 15: Exemplo do Código sobre Candlestick

Interpretação do Gráfico

O gráfico gerado contém elementos visuais que indicam informações sobre a cotação dos ativos:

- **Cores das velas:**
  - Verde: indica alta no preço do ativo (fechamento superior à abertura).
  - Vermelho: indica queda no preço do ativo (fechamento inferior à abertura).
- **Linhas verticais (shadows/wicks):**
  - Representam o intervalo entre o preço máximo e mínimo do dia.
- **Caixa do candlestick:**
  - Indica a diferença entre os preços de abertura e fechamento.
  - Se a caixa for verde, a base representa a abertura e o topo o fechamento.
  - Se a caixa for vermelha, o topo representa a abertura e a base o fechamento.

Ajustes no Gráfico

Por padrão, o Plotly adiciona um range slider na parte inferior do gráfico. Para remover esse controle, basta incluir a seguinte configuração:

```
fig.update_layout(xaxis_rangeslider_visible=False)
```

Caso prefira manter essa funcionalidade ativada, pode ser ajustada para `True`:

```
fig.update_layout(xaxis_rangeslider_visible=True)
```

Isso permite navegar pelo gráfico e visualizar intervalos de tempo específicos.

Conclusão

O candlestick é uma ferramenta essencial para análise financeira, sendo amplamente utilizado no mercado financeiro para identificar tendências e padrões de negociação. Com o Plotly, sua implementação se torna simples e flexível.

10. Mesh 3D

Os gráficos do tipo Mesh 3D no Plotly permitem a criação de superfícies tridimensionais interativas, que podem ser visualizadas de diferentes ângulos. Esse tipo de gráfico é útil quando se deseja representar três dimensões explicitamente, algo que não é possível em histogramas bidimensionais.

Criando um Gráfico Mesh 3D

Para criar um gráfico Mesh 3D, é necessário definir três vetores com valores aleatórios e amplitudes diferentes. No exemplo abaixo, utilizamos vetores de tamanho 70:

```
import numpy as np
import plotly.graph_objects as go

# Definição dos vetores X, Y e Z
N = 70
X = np.random.rand(N) * 70
Y = np.random.rand(N) * 55
Z = np.random.rand(N) * 70

# Criando a figura Mesh 3D
fig = go.Figure(data=[go.Mesh3d(x=X, y=Y, z=Z, opacity=0.5)])

# Exibindo o gráfico
fig.show()
```

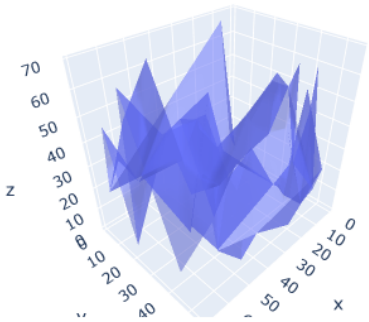

Figure 16: Exemplo Gráfico 3d

Explicação do Código

- **Geração dos dados**: Os vetores X, Y e Z são criados com valores aleatórios e multiplicados por constantes para ajustar suas amplitudes.
- **Criação do gráfico**: Utiliza-se `go.Mesh3d` para definir a malha tridimensional.
- **Opacidade**: O parâmetro `opacity=0.5` torna a superfície semi-transparente, facilitando a visualização.
- **Exibição**: `fig.show()` renderiza o gráfico interativo.

Personalização do Gráfico

O Plotly permite personalizar diversas características do gráfico, como cores. No exemplo abaixo, alteramos a cor da superfície utilizando um código RGB:

```
fig = go.Figure(data=[go.Mesh3d(x=X, y=Y, z=Z, opacity=0.5, color='rgb(255,9,18)')])
fig.show()
```

Isso altera a cor da malha para vermelho (`rgb(255,9,18)`). Outras cores podem ser usadas conforme a necessidade.

Aplicações do Mesh 3D

O Mesh 3D pode ser utilizado para diferentes representações visuais, como:

- **Mapeamento topográfico**: Representação de terrenos utilizando latitude, longitude e altura.
- **Simulação de superfícies matemáticas**: Modelagem de equações tridimensionais.
- **Visualização científica**: Representação de dados experimentais em 3D.

Conclusão

O gráfico Mesh 3D é uma ferramenta poderosa para visualizações tridimensionais interativas no Plotly. Ele segue a mesma estrutura de outros gráficos da biblioteca, facilitando sua utilização. Na próxima aula, será abordado o **Plotly Express**, uma abordagem simplificada para criação de gráficos no Plotly.

11. Plotly Express

Introdução

O Plotly Express é um módulo simplificado da biblioteca Plotly, projetado para facilitar a criação de visualizações gráficas com menos linhas de código. Embora ofereça praticidade, ele também pode limitar a capacidade de customização.

Os criadores da biblioteca recomendam que se inicie pelo Plotly Express devido à sua facilidade de uso. No entanto, o método tradicional da Plotly proporciona maior controle sobre os gráficos.

Importação

O Plotly Express já está incluído na biblioteca Plotly e pode ser acessado da seguinte forma:

```
import plotly.express as px
```

Exemplo de Gráfico Scatter (Dispersão)

Neste exemplo, utilizamos um conjunto de dados embutido na biblioteca, que contém informações sobre medidas físicas de plantas:

```
import plotly.express as px

df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species")
fig.show()
```

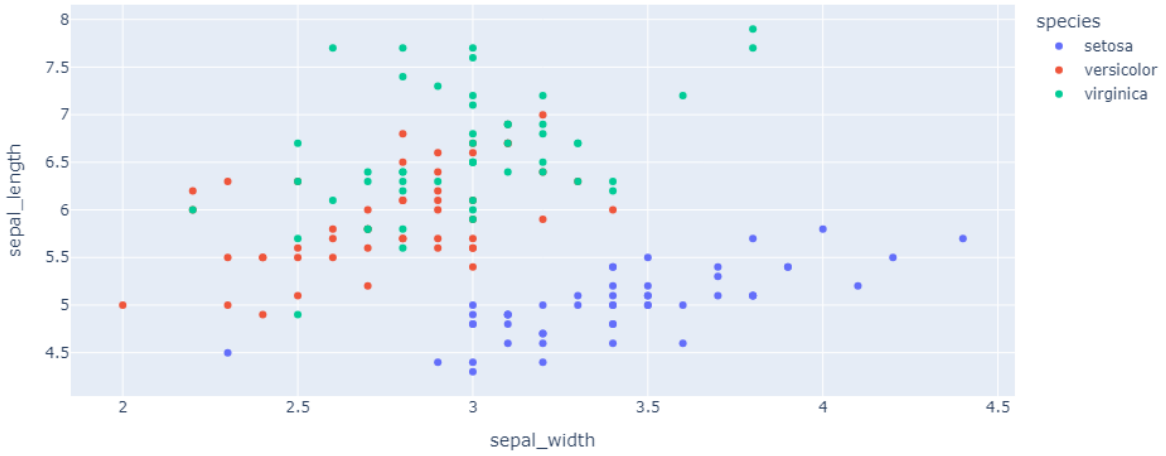

Figure 17: Exemplo de Gráfico com Plotly Express

O comando acima gera um gráfico de dispersão onde os pontos são coloridos de acordo com a espécie da planta.

Gráfico de Barras

Para representar a distribuição de gorjetas em um restaurante fictício:

```
df = px.data.tips()
fig = px.bar(df, x="sex", y="total_bill", color="smoker")
fig.show()
```

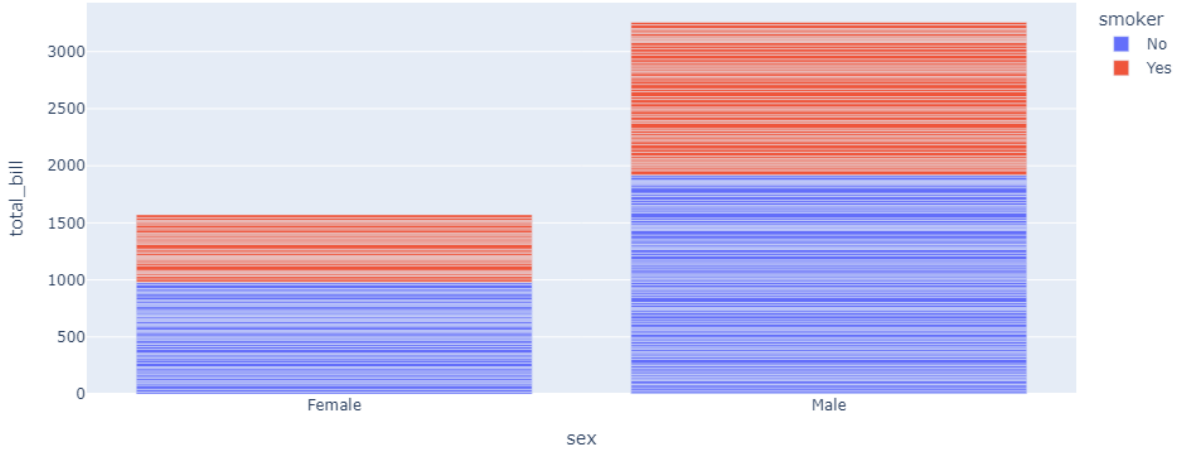

Figure 18: Exemplo de Gráfico com Plotly Express 1

Este gráfico exibe a relação entre gênero e o valor total da conta, diferenciando fumantes e não fumantes por cor.

Scatter Matrix (Matriz de Dispersão)

A scatter matrix permite correlacionar todas as variáveis de um DataFrame:

```
fig = px.scatter_matrix(df, dimensions=["total_bill", "tip", "size"], color="sex")
fig.show()
```

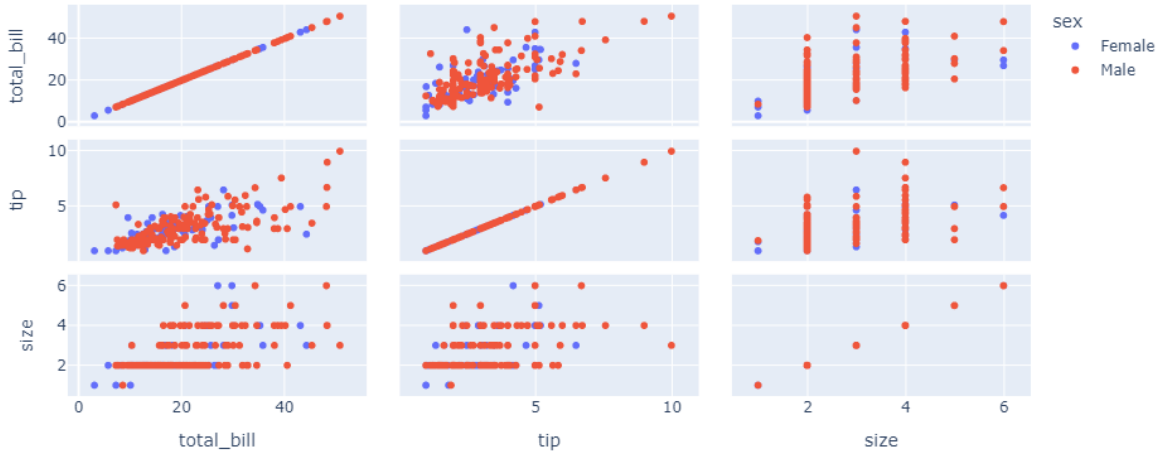

Figure 19: Exemplo de Gráfico com Plotly Express 2

Este comando gera um conjunto de gráficos comparando as variáveis entre si.

Gráfico de Bolhas

Usando dados do GapMinder, pode-se criar um gráfico de bolhas:

```
df = px.data.gapminder()
df_2007 = df[df["year"] == 2007]
fig = px.scatter(df_2007, x="gdpPercap", y="lifeExp", size="pop", color="continent",
↪  log_x=True)
fig.show()
```

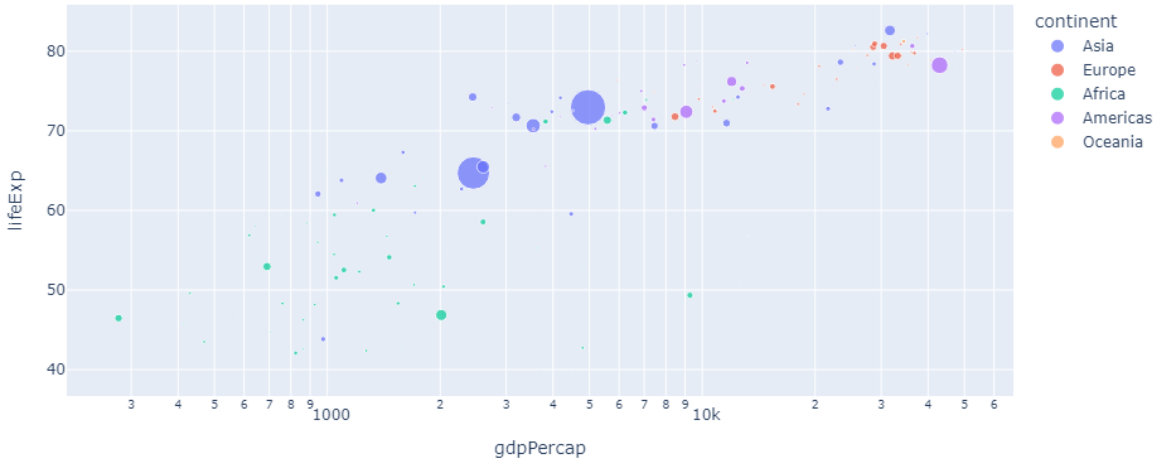

Figure 20: Exemplo de Gráfico com Plotly Express 3

O tamanho das bolhas representa a população de cada país.

Animação Temporal

Para visualizar a evolução de PIB per capita e expectativa de vida ao longo do tempo:

```
fig = px.scatter(df, x="gdpPercap", y="lifeExp", size="pop", color="continent",
↪  animation_frame="year", log_x=True)
fig.show()
```

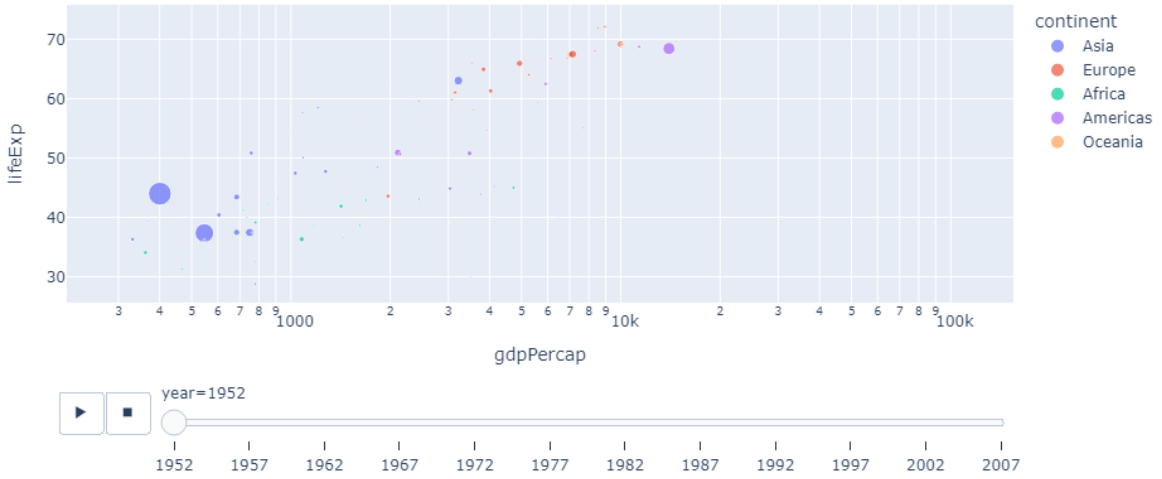

Figure 21: Exemplo de Gráfico com Plotly Express 4

Os dados são apresentados em forma de animação, mostrando as mudanças ao longo dos anos.

Sunburst (Explosão Solar)

Para criar um gráfico hierárquico:

```
fig = px.sunburst(df, path=["continent", "country"], values="pop", color="lifeExp")
fig.show()
```

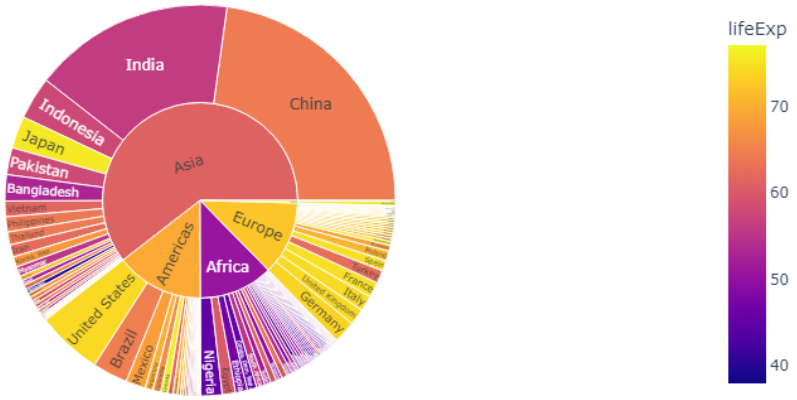

Figure 22: Exemplo de Gráfico com Plotly Express 5

Esse gráfico permite explorar a estrutura de dados hierárquicos.

Heatmaps (Mapas de Densidade)

```
df = px.data.tips()  # Certificando que estamos usando o dataset correto
fig = px.density_heatmap(df, x="total_bill", y="tip", z="size")
fig.show()
```

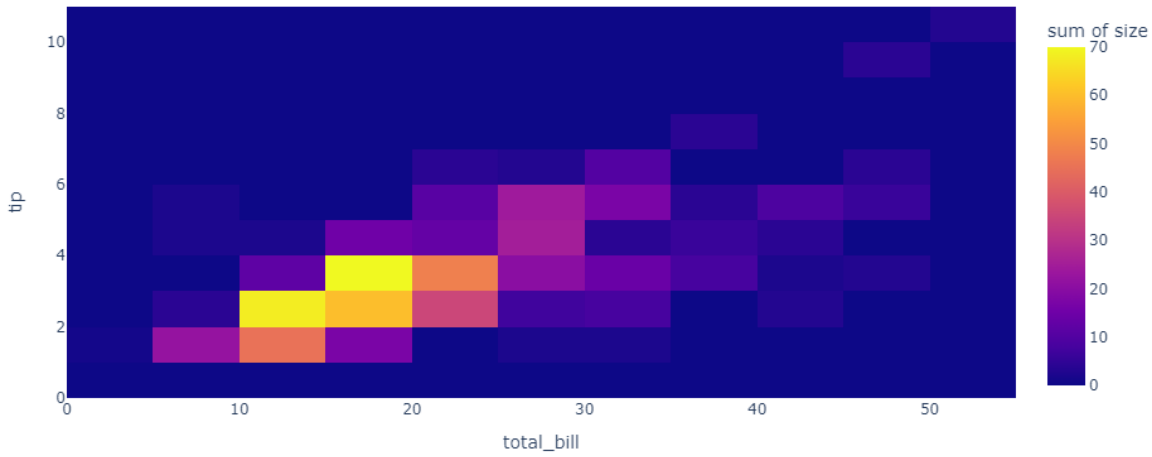

Figure 23: Exemplo de Gráfico com Plotly Express 6

Mapas Interativos

Gráficos geográficos também podem ser criados, como um mapa de expectativa de vida por país:

```
df = px.data.gapminder()  # Garantindo que estamos com os dados corretos antes do mapa
fig = px.choropleth(df, locations="iso_alpha", color="lifeExp", animation_frame="year",
↪  projection="natural earth")
fig.show()
```

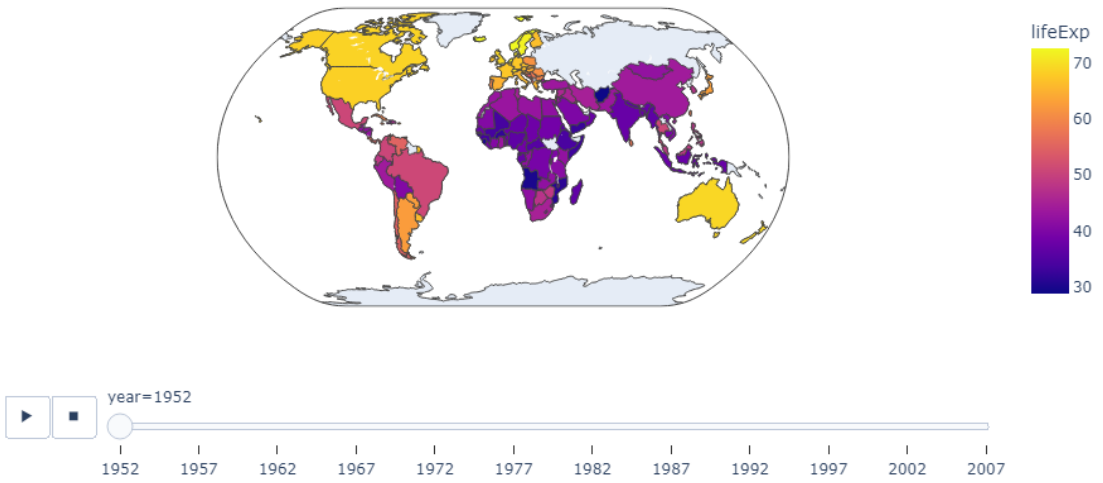

Figure 24: Exemplo de Gráfico com Plotly Express 7

Conclusão

O Plotly Express simplifica a criação de gráficos interativos, sendo ideal para análises rápidas. Entretanto, para personalizações avançadas, é recomendável utilizar o método tradicional do Plotly.

A documentação oficial pode ser consultada para mais detalhes e exemplos.